BIL 322 · FINAL PROJECT · PHASE 1 LAB

Autonomous Drone Swarm Choreographers

Threads, queues, locks, barriers, and one captain whose battery dies.



THE PROJECT

A swarm of drones that dances to music

Hundreds of LED drones perform a choreography, then form a picture in the sky. Four roles make the show.

Center

Artistic Director

Reads the score and the target image, sets each section’s theme, runs the beat clock, hosts the slow planner.

Leaders

Dance Captains

One process per zone. Each asks for a plan, checks it, and directs its troupe.

Followers

Performers

Threads inside a leader in Phase 1. Each flies to a cell, lights a color, reports.

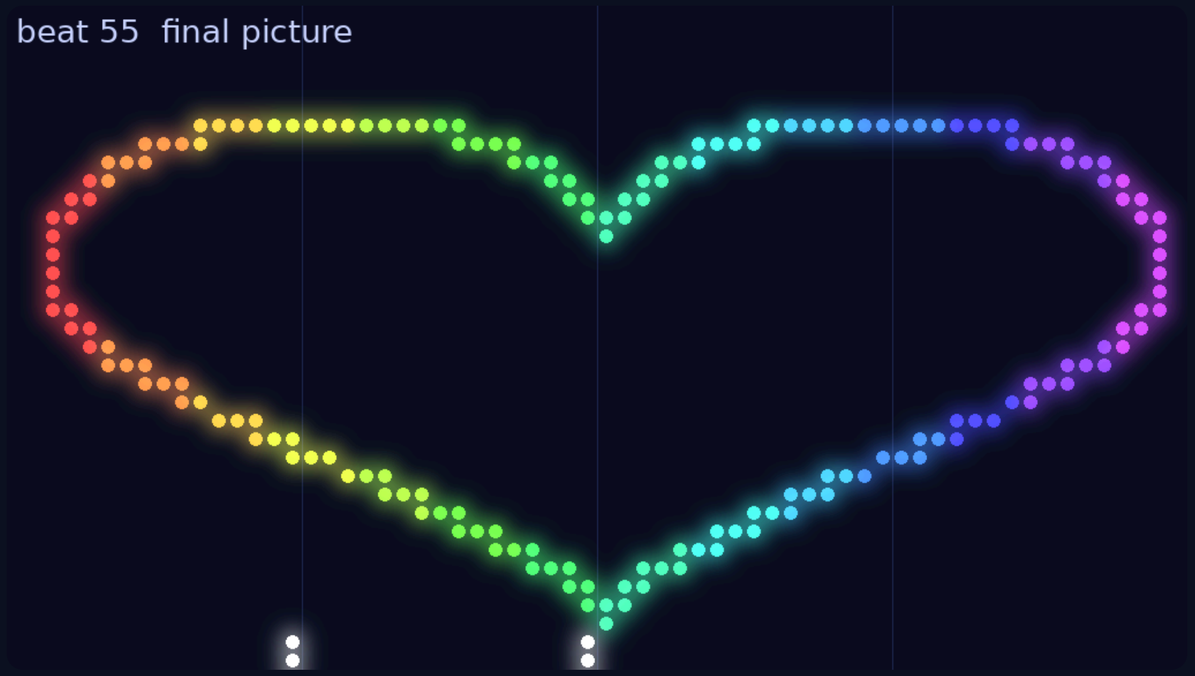
Viewer

Broadcast Booth

A separate GUI from Phase 2. It may drop frames, never slow the swarm.

THE GOAL

What it looks like when it works



4 captains × 46 performers = **184 drones**. Every dot is a drone, every pixel of the heart gets one.

The 4 white dots at the bottom are spare drones that stay home.

```
./center --score data/scores/show.score --followers 46 --ascii 8
```

Phase 2 draws this in a real window.

THREE PHASES, ONE SYSTEM

Same protocol, three transports

	Phase 1 (this week)	Phase 2	Phase 3
Leader	child process (fork, exec)	child process	client process
Performer	thread in its captain	thread in its captain	separate process
Links	pipes	pipes + shared memory	TCP sockets
Viewer	text in the terminal	own process, shared memory	socket client
Captain dies	center notices and restarts it	+ its cells are released	orphans are rehomed

Write your send and receive code against **file descriptors**, and keep parsing separate from transport: Phase 3 then changes the transport, not the program.

PHASE 1 ARCHITECTURE

Two kinds of process, many threads

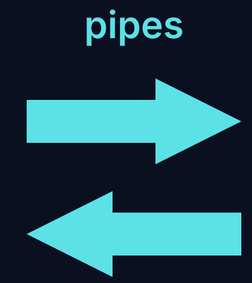
center

one process, the parent

main thread · clock loop: THEME, BEAT, waits for READY and ACK, audits the stage

reader threads · one per leader, parse its pipe

planner pool · threads behind a bounded request queue, Lab 6 pool + arena



leader × 4

one child process per zone: fork, dup2, exec

main thread · reads the center’s lines

thinker · PLAN_REQ, validate, READY

follower threads · F0 … Fk, one barrier per beat, claim cells in the zone’s occupancy grid

THE PROTOCOL (FROZEN)

One message per line, plain text

center → leader

`ASSIGN id x0 y0 x1 y1 n`	your zone
`THEME sec start end theme mood`	coarse direction
`PLAN req n`	then n keyframe lines
`BUSY req retry_ms`	queue full, later
`BEAT n`	start beat n
`SHUTDOWN`	bye

leader → center

`HELLO id n`	I am here
`PLAN_REQ req sec n seed`	give me a plan
`READY id sec`	plan is loaded
`POS id beat f x y c`	where f is
`ACK id beat`	beat done

- Keyframe line: **`beat follower x y color`**. A plan has one for every beat and every follower.
- Unknown message: log and ignore. Malformed: reject, never crash. Lines are at most 256 bytes.
- You can test a leader by typing these lines into its stdin.

HOW IT WORKS

One beat, step by step

1. Center sends **BEAT b** (only after the ACK for b−1).
2. Each follower finds its target for b in the stored plan (no plan: hold).
3. **Release** the cell you are in, then wait at the **barrier**.
4. **Claim** the target cell in the grid, then the **barrier** again.
5. The thread the barrier names sends the **POS** lines and **ACK b**.
6. Center audits collisions and speed, then sends BEAT b+1.

Why two barriers?

In one beat, follower A may move into the cell follower B is leaving.

All releases must happen before any claim, or A's claim fails by accident.

A failed claim is a safety violation, never something to retry in a loop.

WHAT YOU HAVE TO DO

Six tasks in the skeleton

#	Task	Where	Points
1	Bounded queue	`queue.c`	20%
2	Choreographer and captain	`center.c, leader.c (thinker)`	25%
3	Performers and the grid	`grid.c, leader.c (follower)`	25%
4	Find the deadlock	`grid_swap, swapstress`	10%
5	Captain down	`center.c (on_leader_exit)`	10%
6	Tests and report	`make test, make tsan, REPORT.md`	10%

Given: protocol parser, planner, validator, loaders, the clock loop, the unit tests and the test script. **Yours:** the TODO blocks.

TASK 1 · 20%

The bounded queue

```
queue_t *queue_create(size_t cap, size_t item_size);
int queue_push(queue_t *q, const void *item); // blocks if full
int queue_try_push(queue_t *q, const void *item); // -1 if full
int queue_pop(queue_t *q, void *out); // blocks if empty
void queue_close(queue_t *q); // wake everyone
size_t queue_size(queue_t *q); void queue_destroy(queue_t *q);
```

Used twice: the center's **request queue** (try_push, then BUSY if full) and the leader's queue of themes.

Rules

- **while**, not if, around every pthread_cond_wait
- After close: pop drains the rest, then returns −1; push returns −1
- close wakes **everyone**: broadcast
- No busy-waiting, no sleep

TASK 2 · 25%

The choreographer and the captain

Center

handle_plan_req, serve_request

- try_push; queue full: **BUSY 50**. A reader thread never blocks
- **Semaphore**: at most --max-inflight planner calls
- KV blocks from the Lab 6 pool. The pool is **not thread-safe**: wrap it in pool_mu
- Pool empty: BUSY. That is backpressure too
- Every exit path gives back the blocks and posts the semaphore

Captain

thinker_main

- Per THEME: send **PLAN_REQ**, wait for PLAN or BUSY
- BUSY: sleep retry_ms + a **random** extra. A fixed delay makes every leader retry together: livelock
- Run **validate_plan**. Invalid: new seed, at most 6 tries (--flaky 50 tests this)
- Always send **READY**, even after giving up: the troupe holds and the show goes on

TASK 3 · 25%

Performers in lockstep, and the grid

The occupancy grid

grid_claim, grid_release, grid_owner

- One drone per cell, per leader’s zone
- Two modes: **one mutex** for the grid, or **one mutex per row**
- A reader must lock too. Why?
- make bench times both with 1, 2, 4, 8 threads. Put the table in the report and **explain** it

follower_main

for every beat b, in order

```
wait for BEAT b
target = plan or hold
release(old cell)
barrier
claim(target)
barrier
serial thread: POS lines, ACK
```

Failed claim = safety violation: count it, log it, stay where you were.

TASKS 4 AND 5 · 10% EACH

A deadlock to find, a captain to lose

4 · Find the deadlock

```
./swapstress row 8 2
```

- grid_swap locks two rows. In row mode it **hangs**
- Show the stuck threads: gdb -p, thread apply all bt
- Name the Coffman condition you break; fix it with a lock ordering
- Explain why your fix cannot livelock

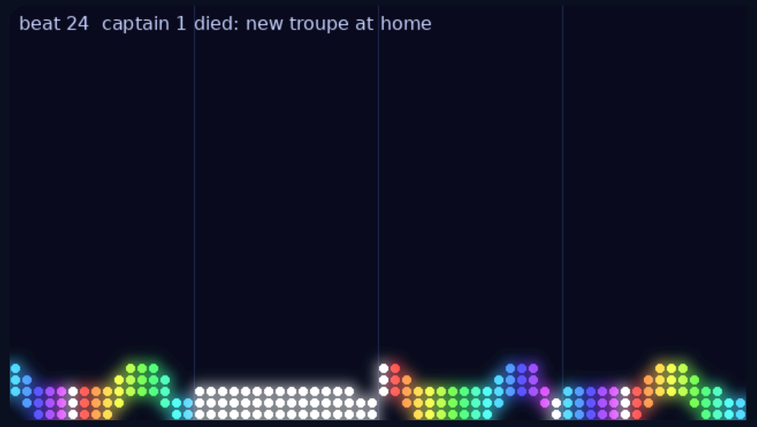
5 · Captain down

```
./center --battery 1:20
```

- Notice by **EOF on the pipe**, not by polling. Print “captain 1 went dark”
- **waitpid**, close the pipes: no zombie
- Wake the clock loop (st_cv) so it stops waiting
- Restart: ASSIGN, THEME from the next beat; the troupe starts at home

TASK 5, AS IT SHOULD LOOK

Captain 1 dies, the show goes on



Beat 24: the second troupe is back at home (white): a new captain, new threads.



Beat 25: the new captain’s plan starts, and the troupe dances with the others.



Beat 55: the picture still forms, with zero collisions and zero speed violations.

GETTING STARTED

Build, test, and the order to work in

```
cd labs/swarm-phase1-skeleton
make # builds, TODOs are empty
make test # PASS/FAIL per check
make tsan # no WARNING allowed
make bench # one lock vs row locks
./center --tempo 0 --ascii 8
```

The full show needs Tasks 1 to 3 together. Until then test each piece alone: **test_queue**, **swapstress**, **gridbench**, or type protocol lines into **./leader --id 0**.

Suggested order

1. queue.c
2. grid claim, release, owner
3. center: serve_request
4. leader: thinker
5. leader: followers
6. captain restart
7. the deadlock fix

HAND-IN AND GRADING

What to submit, what costs you points

Submit

- Your repository (GitHub Classroom) and the link on Google Classroom
- **make test** and **make tsan** pass; Valgrind is clean
- At least **two tests of your own** in tests/run_tests.sh
- **REPORT.md**: architecture sketch, what each mutex protects, lock ordering, the deadlock write-up, captain-down handling, the bench table, AI assistance

Zero tolerance

- Busy-waiting
- An **if** around pthread_cond_wait
- An unprotected Lab 6 pool
- Zombie processes after a captain dies
- A hang on shutdown
- A missing report

Bonus up to +10%: priority queue, reader-writer lock for the grid, battery counter with a charging-station semaphore.

WHEN IT HANGS

Four traps, and how to see them

A barrier nobody reaches

The barrier is shared by **all** followers. One early return or continue between the two waits, and everyone else waits forever.

A lock never released

Every unlock must happen on every path. Early returns are the usual leak, in the queue and in serve_request.

The wrong lock order

In the center: wlock first, then st_mu. The same rule fixes grid_swap: always the lower row first.

Hiding a race with sleep

Never. Run make tsan instead, and read the two stacks it prints.

A hang prints **center: TIMEOUT** after --timeout seconds. Attach with `gdb -p PID`, then `thread apply all bt`.